Fixed-Embedding Experiment: Does Embedding Geometry Determine Transformer Performance?

Claude & C. L.

February 10, 2026

1 Motivation

In the cylinder graph HMM setup, the vocabulary has $|V| = 48$ tokens embedded in $d_{\text{model}} = 8$ dimensions—a 6:1 overcomplete ratio. The learned embedding matrix $W_E \in \mathbb{R}^{48\times 8}$ (tied with the unembedding $W_U = W_E^\top$) is trained jointly with all transformer blocks. Analysis of trained models (cf. `cylinder_hmm_model_comparison.tex`) reveals that W_E consistently develops a 3×3 block structure in cosine similarity, mirroring the three depth-level clusters.

This raises a question: *is the specific learned embedding geometry essential, or does any reasonable embedding suffice?* If the transformer blocks are powerful enough to compensate, then freezing W_E at initialisation should incur little loss. Conversely, if the embedding geometry is critical, then poorly initialised embeddings will create an irreducible performance gap.

2 Experimental Conditions

We freeze the embedding W_E and unembed $W_U = W_E^\top$ at initialisation and train only the transformer blocks (attention layers, MLP layers, and any bias terms). Five initialisation strategies are compared:

1. **Trained** (`trained`). Load W_E from a fully trained checkpoint. This is the *oracle* condition: the embedding already has the "correct" geometry. Any performance gap relative to the jointly-trained baseline reflects the cost of freezing per se (e.g. the inability to fine-tune embedding norms or small rotations).

2. **Random unit-norm** (`random`). Each row of W_E is drawn i.i.d. from $\mathcal{N}(0, I_d)$ and normalised to the unit sphere S^{d-1}. In $d = 8$ dimensions, random unit vectors are approximately orthogonal (expected pairwise cosine similarity ≈ 0), so tokens are roughly equidistant in embedding space with no cluster structure.

3. **Coulomb (Thomson problem)** (`coulomb`). Place 48 points on S^7 by minimising the Coulomb energy
$$E = \sum_{i<j} \frac{1}{\|\mathbf{x}_i - \mathbf{x}_j\|}, \tag{1}$$
via gradient descent with projection back onto the sphere after each step. This produces the most uniformly spaced configuration on S^7, maximising the minimum pairwise distance. Like `random`, this has no cluster structure, but the spacing is deterministic and maximally uniform.

4. **Sparse binary** (`sparse_binary`). Each row has k entries set to 1 (at random positions) and the rest set to 0, then normalised to unit norm. With $k = 3$, there are $\binom{8}{3} = 56 > 48$ distinct patterns, so all rows can be unique. The resulting embedding is sparse and combinatorial, providing a discrete "code" for each token.

5. **PMI-based (SVD)** (`pmi`). Compute the positive pointwise mutual information (PPMI) matrix from sliding-window co-occurrence statistics of the training data:

$$\mathrm{PPMI}(i, j) = \max\bigl(\log \tfrac{P(i,j)}{P(i)\,P(j)},\ 0\bigr). \tag{2}$$

   Then take the truncated SVD: PPMI $\approx U\Sigma V^\top$, and set $W_E = U_{:,:d}\,\Sigma_{:d}^{1/2}$. This embedding directly encodes the statistical structure of the token co-occurrence, akin to classical word2vec/GloVe-style embeddings.

Frozen parameter count. With $|V| = 48$ and $d = 8$, freezing W_E, W_U, and b_U removes $48 \times 8 + 48 \times 8 + 48 = 816$ parameters from the optimiser.

3 Expected Outcomes

Condition	Prediction
`trained`	Should match or nearly match the jointly-trained baseline, since the embedding already captures the cluster structure.
`pmi`	Should perform well: the SVD of PPMI naturally recovers the depth-cluster structure because tokens within the same cluster co-occur far more frequently than cross-cluster tokens.
`random`	Moderate performance. The transformer blocks must learn to "decode" an arbitrary embedding, but the approximate orthogonality of random vectors in $\mathbb{R}^8$ may suffice for token discrimination.
`coulomb`	Similar to `random` but with more uniform spacing; may slightly outperform random due to better worst-case separation.
`sparse_binary`	Unclear. The combinatorial structure is very different from what the transformer would learn; performance depends on whether the attention and MLP layers can adapt to the non-smooth geometry.

Metrics.

- Best validation cross-entropy loss (nats/token), compared to the entropy rate $h \approx 2.44$ and the jointly-trained baseline.
- Convergence speed: number of steps to reach within 0.05 nats of the best achieved loss.
- Per-cluster prediction accuracy: does the model learn the depth-level structure even without embedding cluster structure?

4 Usage

All experiments use the standard `base_config.yaml` (4 layers, $d = 8$, 1 head, full architecture, no LayerNorm). Run each condition with:

```
python src/train_fixed_embedding.py --embedding_mode <mode>
```

For the `trained` condition, additionally pass `--checkpoint_path models/cylinderhmm/<run>/best_model.p`